

Gate-Based Quantum Optimization for Discrete Process Design: A Repair-Aware Benchmark

Yirang Park^a and David E. Bernal Neira^a

^a Davidson School of Chemical Engineering, Purdue University, West Lafayette, IN 47907, USA

ABSTRACT

Discrete choices of routes, units, and operating modes form the first combinatorial barrier in process design. We audit what is required to turn samples from gate-based quantum algorithms into interpretable design candidates, using a fully enumerable drug-substance manufacturing instance with 19 binary flow variables. The penalty-based quadratic unconstrained binary optimization (QUBO) reformulation adds 17 auxiliary variables, so a sampled 36-bit state becomes a design candidate only after projection onto the flow variables and exact repair of the auxiliaries. A 19-variable quadratic surrogate of the repaired landscape and quantum approximate optimization algorithm (QAOA) angles optimized offline on that surrogate concentrate probability on the optimal flowsheet (664 of 262144 ideal reads), yet a simple hill climber, an exact solver, and exhaustive enumeration all reach the optimum faster. Sensitivity tests over penalty weights, surrogate training sets, circuit depth, optimizer budget, and random seeds leave these conclusions unchanged, and a small run on IBM hardware verifies the pipeline without adding advantage. Because the surrogate and the transferred angles rely on exhaustive enumeration, which is feasible only at this scale, the study is a workflow-feasibility and interpretation benchmark rather than a claim of quantum speedup.

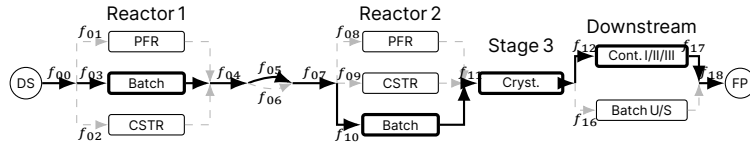
Keywords: quantum computing, process design, discrete optimization, QUBO reformulation, pharmaceutical manufacturing

INTRODUCTION

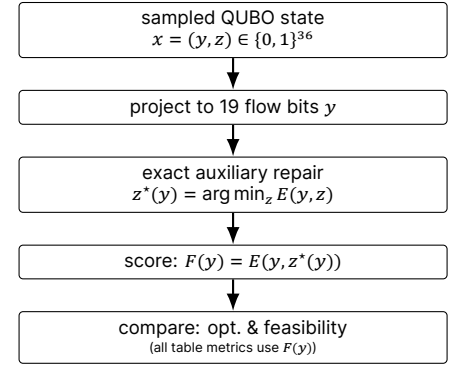
Discrete choices are often the first barrier in process design. Before a flowsheet can be simulated or optimized over continuous variables, the designer must commit to a set of mutually exclusive options: which synthesis route to use, which unit operations to include, and which operating modes to run [1]. In pharmaceutical manufacturing, these choices are particularly consequential because route and equipment selection shape cost, yield, and regulatory burden, and because digital design and optimization tools for this sector are comparatively recent [2, 3]. The resulting models are combinatorial, and even modestly sized superstructures generate large discrete search spaces.

Gate-based quantum optimization is one of the emerg-

ing tools proposed for this setting. A gate-based quantum computer operates on quantum bits (qubits) by applying sequences of parameterized gate operations that form a quantum circuit; measuring the circuit returns a binary string, and the circuit parameters, called angles, are tuned so that low-energy strings appear with high probability. The quantum approximate optimization algorithm (QAOA) [4] and the variational quantum eigensolver (VQE) [5, 6] follow this variational paradigm: QAOA uses p alternating cost and mixer layers, and VQE uses a parameterized ansatz circuit (here, a hardware-efficient ansatz); in both cases, a classical optimizer adjusts the angles. Current hardware is noisy and of intermediate scale [7], and benchmarking surveys stress that a practical quantum advantage for optimization has not yet been established



(a) DS-MFG manufacturing superstructure (19 binary arcs)



(b) Repair-aware scoring pipeline

Figure 1. Overview of the DS-MFG case study. **(a)** Manufacturing superstructure: thick arrows trace the optimal flowsheet (Batch Rxn 1 → Route A (f_{05}) → Batch Rxn 2 → Cryst. → Cont. I (f_{13})); dashed arrows mark the non-optimal alternatives; the backbone arcs $f_{00}, f_{04}, f_{07}, f_{11}, f_{18}$ carry every feasible flow. Route B (f_{06}) is activated only when a continuous Reactor 1 feeds a batch Reactor 2; Cont. I/II/III share entry arc f_{12} and exit arc f_{17} (optimal uses f_{13}). **(b)** Every sampled 36-bit QUBO string is projected to the 19 flow variables, repaired exactly via the 13-component auxiliary decomposition (GitHub repository), and scored against the certified optimum and the 36-member feasible-design pool; hit rates appear in Table 1.

[8]; the relevant question is what is required to use these methods today and how their output compares with classical baselines.

Within process systems engineering, quantum computing has been reviewed as an emerging capability [9], and hybrid quantum-classical strategies have been proposed for discrete optimization in process and energy systems [10]. Discrete-design instances have been mapped to the quadratic unconstrained binary optimization (QUBO) form [11] and solved with Ising-based annealing hardware [10, 12]. These works establish that the mapping is possible; few examine what happens to the samples after the reformulation, and the gate-based path is less explored than annealing. Casting a constrained problem as a QUBO introduces auxiliary variables [13] that are bookkeeping rather than design decisions, so a raw quantum sample is not directly interpretable as a flowsheet. The question is: what does it take to turn gate-based quantum samples into process-design decisions, and how does this compare with classical methods?

We answer this through an end-to-end audit on a single drug-substance manufacturing (DS-MFG) instance, chosen to be small enough to enumerate exactly yet large enough to expose the reformulation and interpretation issues that arise at scale. Our contribution is a repair-aware audit of the full path from a constrained process-design model to usable gate-based quantum samples. We find: (i) raw QUBO samples require projection and exact auxiliary repair before they are interpretable as design candidates; (ii) a quadratic surrogate over the design vari-

ables fits the repaired landscape globally but fails as an optimizer; (iii) QAOA with angles optimized offline on that surrogate concentrates probability on good flowsheets but does not beat classical baselines; (iv) these findings hold across the tested penalty weights, surrogate training sets, circuit depths, optimizer budgets, and seeds; and (v) a hardware run confirms pipeline feasibility without adding advantage. Because the surrogate and the offline angles are built from exhaustive repair enumeration, feasible only for this small instance, the workflow is a diagnostic benchmark for sample interpretation, not a scalable optimization shortcut.

METHODS

The Discrete Design Instance

The DS-MFG instance is the discrete subproblem of a drug-substance manufacturing case study introduced by Park and Bernal Neira [12], where a mixed-integer nonlinear design problem is decomposed so that the discrete part is handled by combinatorial solvers and the nonlinear part by simulators. We keep the same discrete instance but ask what is required to run gate-based optimizers. The instance is a binary integer program for capital-expenditure selection in the manufacturing superstructure of Figure 1(a): the 19 binary variables $f_i, i \in F$, encode candidate material-flow arcs and unit-operation selections, and the objective is a normalized capital-cost score with arc costs c_i (lower values indicate lower capital expenditure; scaled as in [12]). The discrete problem has

the form

$$\begin{aligned}
\min_f \quad & \sum_{i \in F} c_i f_i \\
\text{s.t.} \quad & \sum_{i \in \text{In}(n)} f_i = \sum_{j \in \text{Out}(n)} f_j \quad \forall n \in N, \\
& f_{00} = 1, \quad f_{18} = 1, \\
& (1 - f_{01}) + (1 - f_{10}) + f_{06} \geq 1, \\
& (1 - f_{02}) + (1 - f_{10}) + f_{06} \geq 1, \\
& (1 - f_{03}) + (1 - f_{10}) + (1 - f_{06}) \geq 1, \\
& f_{01} + f_{02} + f_{05} \geq 1, \quad f_{05} + f_{10} \geq 1, \\
& f_{17} \geq f_k, \quad k \in \{13, 14, 15\}, \\
& f_i \in \{0, 1\} \quad \forall i \in F.
\end{aligned} \tag{1}$$

Eq. (1) contains three constraint classes: flow balance at each node $n \in N$, a fixed source arc f_{00} and sink arc f_{18} , and logic constraints that enforce a valid flowsheet (the Route B activation rule, the reactor-route compatibility rules, and the shared exit arc of the continuous downstream options).

Solved with Gurobi [14], the model (19 binary variables, 20 constraints) returns an optimal capital-cost score of 11.7095; with feasible-solution enumeration enabled (`PoolSearchMode=2`), all 36 feasible flows are retained in a single deterministic solve (0.073 s). We treat 11.7095 as the certified global optimum and use it as the reference throughout.

From Integer Program to QUBO and Back

To run QAOA and VQE, the integer program must be expressed as a QUBO, in which all constraints are folded into a single quadratic objective over binary variables [11, 13]. Inequality constraints require auxiliary (slack) variables; the penalty weights and auxiliary count are not unique, and both affect how faithfully the QUBO represents the original problem [13]. For this instance, the penalty reformulation introduces 17 auxiliary variables, so the QUBO acts on 36 binary variables. For a bitstring $x \in \{0, 1\}^{36}$, the encoded objective is

$$E_{\text{QUBO}}(x) = \ell^T x + x^T Q x + b, \tag{2}$$

with linear coefficients ℓ , quadratic coefficient matrix Q , and constant offset b . The 36 variables separate into the original flow variables and the auxiliaries, $x = (y, z)$, $y \in \{0, 1\}^{19}$, $z \in \{0, 1\}^{17}$. Only y encodes a design decision; z is bookkeeping, so a sampled 36-bit string does not map directly to a flowsheet. We make it interpretable by projecting onto y and *repairing* the auxiliaries, that is, choosing the assignment that

minimizes the QUBO energy for that flow,

$$F(y) = \min_{z \in \{0, 1\}^{17}} E_{\text{QUBO}}(y, z). \tag{3}$$

The repair is exact and cheap here because, once y is fixed, the auxiliary-auxiliary coupling graph splits into 13 independent connected components: 9 singletons (one auxiliary bit each) and 4 pairs (two auxiliary bits each). The singletons correspond to the at-most-one inequality constraints for each decision group and to the downstream flow-balance conditions; the pairs arise from product-linearization inequalities for the Route B activation rule ($f_{06} = 1$ iff Reactor 1 is continuous and Reactor 2 is batch). Each component is solved by enumeration over its 2 or 4 assignments, so $9 \times 2 + 4 \times 4 = 34$ local evaluations replace enumeration of $2^{17} = 131,072$ auxiliary states. The full component table, with auxiliary indices, flow neighbors, and constraint roles, is recorded in the shared code repository (link in Acknowledgments). The decomposition is specific to this instance (see Discussion).

The repaired minimizer over all 2^{19} flows recovers the optimum, and the repaired QUBO scores match the integer-program objective to within 3.1×10^{-13} , confirming that they encode the same objective. Exact enumeration finds exactly 36 feasible flows, as does the solution pool of Gurobi, and they occupy repaired ranks 1–36.

The repaired objective $F(y)$ is not a quadratic function of y , but QAOA and VQE require a QUBO input. Rather than using the full 36-variable QUBO, which mixes design variables with auxiliaries and requires 36-qubit circuits, we construct a reduced 19-variable QUBO over the design space y by fitting a quadratic surrogate to the repaired landscape,

$$\tilde{F}(y) = \hat{b} + \hat{\ell}^T y + y^T \hat{Q} y, \tag{4}$$

by unweighted least squares over all 2^{19} repaired flows ($R^2 \approx 0.999$). The surrogate serves only as a compact 19-qubit sampling Hamiltonian; it is never used to certify solution quality, and every reported metric is evaluated after exact repair under F .

Quantum, Classical, and Hardware Workflows

The local QAOA and VQE workflows run through `QiskitOpt.jl`, a Julia wrapper for Qiskit [15], with QUBO models constructed via the `QUBO.jl` ecosystem [13]. Increasing p makes QAOA more expressive and can be viewed as a finer discretization of an adiabatic path [16], but deeper circuits are more sensitive to noise on current NISQ devices [7]. Direct full-QUBO QAOA and VQE operate on the 36-variable formulation, and reduced-surrogate QAOA and VQE on the 19-variable surrogate; both tune their angles with the native COBYLA optimizer of Qiskit and are scored by exact repair. Surrogate VQE

uses an `EfficientSU2` ansatz. The best-performing workflow, which we call *energy-targeted QAOA*, replaces the native angle search by an offline step: the angles are optimized on the surrogate with `JuliQAOA` [17], which minimizes the expected surrogate energy $\mathbb{E}_{y \sim p_\theta}[\hat{F}(y)]$ by basin hopping with BFGS local optimization [6], and are then transferred to a Qiskit circuit for sampling. The offline search cost is excluded from the call times in Table 1. As classical baselines, we sample the repaired 19-flow objective by uniform random sampling and by single-bit-flip hill climbing with random restarts. As a hardware feasibility test, we run a fixed-angle QAOA circuit on the IBM `ibm_fez` quantum computer; its angles were optimized offline to concentrate probability on the ten lowest-cost flows rather than on the expected energy.

For each sampled state $x = (y, z)$ we record the raw QUBO energy $E_{\text{QUBO}}(y, z)$, the repaired objective $F(y)$, and the auxiliary gap. An *optimum hit* means that the repaired flow equals the certified optimal flowsheet, the unique minimum-cost feasible design; a *feasible-design hit* means that it matches any of the 36 feasible flows. We report both hit rates with two-sided Wilson 95% intervals; zero-hit rows show the upper endpoint. The empirical $\text{TTS}_{99} = \tau[\log(0.01)/\log(1 - \hat{p})]$ uses the recorded call time per sample τ ; zero-hit rows report ∞ . These values compare sampling costs and exclude preprocessing. Evaluations within a search run are not independent trials, and VQE performance varies with the seed, so the intervals and TTS values are descriptive and do not guarantee success across repeated runs.

RESULTS

Table 1 summarizes the primary outcomes: energy-targeted QAOA gives the highest stochastic feasible-design rate, but hill climbing and Gurobi remain faster on the recorded cost metrics.

Repair Changes the Meaning of a Sample

Sampling the 36-variable QUBO directly does not yield an interpretable design result. A $p = 2$ full-QUBO QAOA found no optimum in 512 samples; a 32768-read run returned it only 4 times after repair, and never as a fully optimal 36-bit string without projection. Without projection and exact repair, a raw QUBO sample cannot be read as a flowsheet; Figure 1(b) summarizes the scoring pipeline.

Reduction and Offline Optimization Concentrate QAOA Samples

Once the pipeline is in place, reduction to the surrogate and the offline angle search are what make QAOA sam-

ples concentrate on feasible, optimal flowsheets. Energy-targeted QAOA at $p = 5$ sampled the certified optimum 664 times in 262144 ideal reads and matched a feasible flowsheet 32502 times (12.4%, CI [12.3, 12.5]%), the highest feasible-design rate of any workflow; an independent state-vector calculation puts the optimum probability of this circuit at 2.63×10^{-3} , consistent with the sampled 2.53×10^{-3} . The near-perfect global R^2 of the surrogate did not imply optimization fidelity: its own minimizer repairs to 14.5505 rather than 11.7095, the true optimum ranks only 33rd under the surrogate, and among the 50 lowest-cost repaired flows the absolute surrogate errors reach 29.4. Refitting the surrogate to uniform random subsets of 1024 to 65536 flows (three seeds per size) changes neither its R^2 (0.9988–0.9991) nor this failure (minimizers repair to 14.55–18.18), so the failure is a property of the quadratic form rather than of the training set.

VQE and Classical Baselines Bound the Claim

VQE benefits from the same reduction but produces a flatter distribution. Full-QUBO VQE found no optimum in a five-seed sweep. Table 1 reports all 20 initial reduced-surrogate runs, seeds 74001–74020 with 32768 reads each, with 3 optimum and 61 feasible-design hits in 655360 reads; only seeds 74001 and 74007 found the optimum (1 and 2 hits). Hill climbing found the optimum 19 times in 262144 evaluations; despite a lower hit rate than QAOA, its much smaller per-sample cost gives a better $\text{TTS}_{99}\text{-opt}$ (0.23 s vs. 0.37 s for QAOA). Random sampling never found the optimum. Thus, QAOA gives the highest stochastic feasible-design rate but does not beat classical baselines on time-to-solution.

Hardware Execution Is an Implementation Feasibility Check

To check that the implementation runs on hardware, we submitted the fixed-angle QAOA circuit to the `ibm_fez` quantum computer across three repeats and three random transpile seeds (92001–92003), with 4096 measurement shots per repeat, for 36864 total hardware reads (9 jobs, 73.5 s elapsed including cloud queue). The run produced 4 feasible-design reads and no optimum read; the best hardware sample was the second-cheapest feasible flowsheet (objective 11.8105). Using this recorded time gives a $\text{TTS}_{99}\text{-feas}$ of 84.6 s, versus 0.006 s for hill climbing and 0.073 s for the deterministic Gurobi solve. Queue time was not recorded separately, so we cannot determine how much it contributed. With zero optimum hits, the Wilson 95% upper bound on the hardware optimum-hit rate is about 1.04×10^{-4} .

Table 1: Primary outcomes after exact auxiliary repair. Opt rate: certified-optimum hit rate; feas rate: any of the 36 feasible designs. Wilson 95% intervals [lo, hi] use the same units as the rate; zero-hit rows show the upper endpoint. Call time covers the recorded solver call or hardware run. It excludes prior preprocessing and any scoring performed after the call. TTS99 uses these call times and does not measure the full workflow or guarantee success across seeds (Methods). Gurobi reports a deterministic solve time.

Workflow	Call time (s)	Opt rate [95% CI]	Feas rate [95% CI]	TTS99-opt (s)	TTS99-feas (s)
Gurobi solution pool	0.073	Deterministic optimum	36/36 (exhaustive)	0.073	0.073
Uniform random sampling	2.08	$< 1.5 \times 10^{-5}$	$8.0 [5.2, 12.2] \times 10^{-5}$	∞	0.46
Hill-climb restarts	0.93	$7.2 [4.6, 11.3] \times 10^{-5}$	$2.75 [2.56, 2.96] \times 10^{-3}$	0.23	0.006
Energy-targeted QAOA, $p=5$	53.9	$2.53 [2.35, 2.73] \times 10^{-3}$	$0.124 [0.123, 0.125]$	0.37	0.007
Reduced-surrogate VQE, all 20 seeds	45.4	$4.6 [1.6, 13.5] \times 10^{-6}$	$9.3 [7.2, 12.0] \times 10^{-5}$	69.7	3.43
IBM <code>ibm_fez</code> pilot	73.5	$< 1.1 \times 10^{-4}$	$1.1 [0.42, 2.8] \times 10^{-4}$	∞	84.6

Key counts: QAOA $p=5$: 664 opt. hits and 32502 feas. hits in 262144. IBM `ibm_fez`: 0 opt. hits and 4 feas. hits in 36864.

Robustness and Additional Classical Baselines

We tested how the results change with penalty weights, QAOA depth, optimizer budget, and random seeds, and we compared additional classical and annealing baselines. The detailed settings, seeds, results, and timings of each run are not included here but are recorded in the shared repository. For the penalty test, write the QUBO as capital cost $c(y)$ plus penalty $P(y, z)$ and scale only the penalty, $F_\alpha(y) = c(y) + \alpha \min_z P(y, z)$. Among the tested values, $\alpha = 0.01, 0.05, 0.1, 0.25, 0.5, 1, 2$, and 4 , exact enumeration gives the same certified optimum for $\alpha \geq 0.25$; the three smaller weights give infeasible minima. All feasible designs rank ahead of every infeasible design for tested $\alpha \geq 0.5$, and at the submitted weight ($\alpha = 1$) the lowest infeasible score exceeds the highest feasible score by 31.3717. These results apply to this instance with all penalty terms scaled together.

Two further tests concern the offline angle search of energy-targeted QAOA. The first varies circuit depth: repeating the search for $p = 1$ to 5 with three random seeds, the state-vector optimum probability rises from 1.42×10^{-4} at $p = 1$ to $(2.63\text{--}2.81) \times 10^{-3}$ at $p = 5$, consistent with the sampled rate in Table 1. The second varies the optimizer budget at fixed $p = 5$: running the basin-hopping search with five steps (random angle perturbations, each followed by a BFGS local search) instead of a single step leaves the optimum probability at 2.63×10^{-3} . Depth, not optimizer budget, controls how strongly the samples concentrate.

VQE outcomes, in contrast, depend strongly on the seed. Seed 74018, which had the most top-50 hits in the 20-seed screen (23) but no optimum hit, gave 20 optimum hits when extended to 524288 reads, while seeds 74001 and 74007 gave 5 and 3; reruns of the same three seeds

that also retained circuit metadata gave 3, 1, and 0. A single seed therefore cannot represent VQE performance, and Table 1 reports the complete screen.

Finally, we compared simulated annealing [18], uniform sampling, and hill climbing with energy-targeted QAOA at a common budget of 262144 evaluations or reads, using five seeds for each classical method. Annealing found the optimum 9–25 times per run, hill climbing 17–27 times, and uniform sampling at most once, whereas the $p = 5$ QAOA circuit returns it about 690 times at its state-vector probability (664 observed in Table 1). Each classical run took 0.6–2.3 s, against 53.9 s of QAOA sampling preceded by roughly 100 s of offline angle search, so QAOA gives the most concentrated distribution but the slowest route to the optimum. Quantum annealing lies outside the gate-based scope of this paper; our earlier study of the same instance [12] reports that simulated annealing and the D-Wave Advantage2 annealer each recover all 36 feasible designs in a single run of 1000 reads, with 99% time-to-optimum of 197.8 and 199.6 s, while Advantage 4.1 recovers 28 designs (311.9 s of quantum processing time). Those metrics score raw annealer samples on different hardware and read budgets, so they complement Table 1 rather than extend it.

DISCUSSION AND LIMITATIONS

This study asks what is needed to turn QAOA and VQE samples into meaningful process-design decisions. The steps are to separate design and auxiliary variables, repair the auxiliaries exactly, fit a quadratic surrogate, optimize the angles offline, and score samples in the original flow space. Even when the samples concentrate on good flow-sheets, classical methods remain faster for this instance.

Table 1 is not an end-to-end cost comparison. Its call

times cover the sampling stage only, whereas a complete quantum workflow also includes reformulation, exhaustive enumeration and repair, surrogate fitting, offline angle optimization, circuit compilation, and, on hardware, queuing and result retrieval. Those stage times were not recorded for the original runs, and the 73.5 s hardware record does not separate compilation, queuing, and execution, so the quantum cost cannot be totaled. The sampling-only TTS values therefore flatter the quantum rows, and a full accounting could only widen the classical margin.

Exact enumeration finds the optimum before quantum sampling begins, so these experiments test how to interpret samples after repair. Repair is cheap here because the auxiliary graph has 13 small components; for a general QUBO, auxiliary repair can itself be NP-hard. Enumeration and dense state-vector simulation each grow as 2^n , while repair by component enumeration requires $\sum_c 2^{|c|}$ local assignments. Larger problems would need approximations whose effects on feasibility and optimality would have to be quantified. Tests on this one 19-flow instance cannot establish scalability or a total runtime advantage for larger process designs [8].

CONCLUSION

For this drug-substance manufacturing case study, QAOA and VQE samples become interpretable as process-design decisions only after projection to the flow variables and exact auxiliary repair. Energy-targeted QAOA concentrates probability on good flowsheets but does not beat classical baselines on any time-to-solution metric (Table 1), and this conclusion is unchanged across the tested penalty weights, surrogate training sets, circuit depths, optimizer budgets, and seeds. The central lesson is that quantum-sample quality must be evaluated after mapping back to the original process decisions; raw QUBO samples can overstate both feasibility and optimization performance, and honest benchmarks must account for the full preprocessing cost.

ACKNOWLEDGMENTS

This material is based upon work supported by the Center for Quantum Technologies under the Industry-University Cooperative Research Center Program at the US National Science Foundation under Grant No. 2224960. Scripts, data files, circuit records, and the component table of the auxiliary repair are archived in the shared repository at <https://github.com/SECQUOIA/pse-quantum-discrete-optimization>.

DECLARATION OF USE OF AI

The authors used generative AI tools to assist with initial code drafting, data-analysis scripting, and draft visualization design. All code, analyses, visualizations, and manuscript text were reviewed, corrected, and revised by the authors, who take full responsibility for the accuracy and integrity of all results.

AUTHOR IDENTIFIERS

Author ORCIDs:

Park: 0009-0008-6629-3308

Bernal Neira: 0000-0002-8308-5016

REFERENCES

- [1] L. Mencarelli, Q. Chen, A. Pagot, and I. E. Grossmann. "A review on superstructure optimization approaches in process system engineering". In: *Computers and Chemical Engineering* 136 (2020), p. 106808. DOI: [10.1016/j.compchemeng.2020.106808](https://doi.org/10.1016/j.compchemeng.2020.106808).
- [2] D. Casas-Orozco, D. Laky, V. Wang, M. Abdi, X. Feng, E. Wood, C. Laird, G. V. Reklaitis, and Z. K. Nagy. "PharmaPy: An object-oriented tool for the development of hybrid pharmaceutical flowsheets". In: *Computers & Chemical Engineering* 153 (2021), p. 107408. DOI: [10.1016/j.compchemeng.2021.107408](https://doi.org/10.1016/j.compchemeng.2021.107408).
- [3] D. Laky, D. Casas-Orozco, C. D. Laird, G. V. Reklaitis, and Z. K. Nagy. "Simulation-Optimization Framework for the Digital Design of Pharmaceutical Processes Using Pyomo and PharmaPy". In: *Industrial & Engineering Chemistry Research* 61.43 (2022), pp. 16128–16140. DOI: [10.1021/acs.iecr.2c01636](https://doi.org/10.1021/acs.iecr.2c01636).
- [4] E. Farhi, J. Goldstone, and S. Gutmann. *A Quantum Approximate Optimization Algorithm*. 2014. arXiv: [1411.4028](https://arxiv.org/abs/1411.4028) [quant-ph]. URL: <https://arxiv.org/abs/1411.4028>.
- [5] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien. "A variational eigenvalue solver on a photonic quantum processor". In: *Nature Communications* 5 (2014), p. 4213. DOI: [10.1038/ncomms5213](https://doi.org/10.1038/ncomms5213). arXiv: [1304.3061](https://arxiv.org/abs/1304.3061) [quant-ph].
- [6] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles. "Variational quantum algorithms". In: *Nature Reviews Physics* 3 (2021), pp. 625–644. DOI: [10.1038/s42254-021-00348-9](https://doi.org/10.1038/s42254-021-00348-9). arXiv: [2012.09265](https://arxiv.org/abs/2012.09265) [quant-ph].

- [7] J. Preskill. "Quantum Computing in the NISQ era and beyond". In: *Quantum* 2 (2018), p. 79. DOI: [10.2331/q-2018-08-06-79](https://doi.org/10.2331/q-2018-08-06-79).
- [8] A. Abbas, A. Ambainis, B. Augustino, et al. "Challenges and opportunities in quantum optimization". In: *Nature Reviews Physics* 6.12 (2024), pp. 718–735. DOI: [10.1038/s42254-024-00770-9](https://doi.org/10.1038/s42254-024-00770-9).
- [9] D. E. Bernal, A. Ajagekar, S. M. Harwood, S. T. Stober, D. Trenev, and F. You. "Perspectives of quantum computing for chemical engineering". In: *AIChE Journal* 68.6 (2022), e17651. DOI: [10.1002/aic.17651](https://doi.org/10.1002/aic.17651).
- [10] A. Ajagekar and F. You. "Quantum computing for energy systems optimization: Challenges and opportunities". In: *Energy* 179 (2019), pp. 76–89. DOI: [10.1016/j.energy.2019.04.186](https://doi.org/10.1016/j.energy.2019.04.186).
- [11] F. Glover, G. Kochenberger, and Y. Du. *A Tutorial on Formulating and Using QUBO Models*. 2019. arXiv: [1811.11538](https://arxiv.org/abs/1811.11538) [cs.DS]. URL: <https://arxiv.org/abs/1811.11538>.
- [12] Y. Park and D. E. Bernal Neira. *Leveraging Classical and Quantum Computing for Process Systems Engineering Applications: Decomposition Algorithm with Ising Solvers for Efficient Discrete Landscape Exploration*. 2026. arXiv: [2603.19520](https://arxiv.org/abs/2603.19520) [math.OC]. URL: <https://arxiv.org/abs/2603.19520>.
- [13] P. M. Xavier, P. Ripper, T. Andrade, J. D. Garcia, N. Maculan, and D. E. Bernal Neira. *QUBO.jl: A Julia ecosystem for quadratic unconstrained binary optimization*. 2023. DOI: [10.48550/arXiv.2307.02577](https://doi.org/10.48550/arXiv.2307.02577). arXiv: [2307.02577](https://arxiv.org/abs/2307.02577) [math.OC]. URL: <https://arxiv.org/abs/2307.02577>.
- [14] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. 2026. URL: <https://www.gurobi.com>.
- [15] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta. *Quantum computing with Qiskit*. 2024. DOI: [10.48550/arXiv.2405.08810](https://doi.org/10.48550/arXiv.2405.08810). arXiv: [2405.08810](https://arxiv.org/abs/2405.08810) [quant-ph].
- [16] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser. *Quantum Computation by Adiabatic Evolution*. 2000. arXiv: [quant-ph/0001106](https://arxiv.org/abs/quant-ph/0001106). URL: <https://arxiv.org/abs/quant-ph/0001106>.
- [17] J. Golden, A. Bärttschi, D. O'Malley, E. Pelofske, and S. Eidenbenz. "JuliQAOA: Fast, Flexible QAOA Simulation". In: *SC-W '23: Proceedings of the Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. Association for Computing Machinery, 2023, pp. 1454–1459. DOI: [10.1145/3624062.3624220](https://doi.org/10.1145/3624062.3624220). arXiv: [2312.06451](https://arxiv.org/abs/2312.06451) [quant-ph].
- [18] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. "Optimization by simulated annealing". In: *Science* 220 (4598 1983), pp. 671–680. ISSN: 00368075. DOI: [10.1126/SCIENCE.220.4598.671](https://doi.org/10.1126/SCIENCE.220.4598.671). URL: <https://www.science.org>.

© 2027 by the authors. Licensed to PSEcommunity.org and PSE Press. This is an open access article under the creative commons CC-BY-SA licensing terms. Credit must be given to creator and adaptations must be shared under the same terms. See <https://creativecommons.org/licenses/by-sa/4.0/>.

